

Name: GURUBASAVA

USN: 1RUB25CSE0008

SNMP GetRequest Implementation using VIBE coding&LLM

1. Introduction to SNMP

Simple Network Management Protocol (SNMP) is an application-layer protocol used for network management. It allows administrators to monitor network performance, find and solve network problems, and plan for network growth. SNMP consists of a set of standards for network management, including an application layer protocol, a database schema, and a set of data objects. It typically operates on UDP port 161.

2. Explanation of SNMP GET Request

An SNMP GET request is a message sent by an SNMP manager (in this case, our Python script) to an SNMP agent (the device being monitored). The manager uses this request to retrieve the value of one or more variables from the agent's Management Information Base (MIB). The agent responds with an SNMP GET-RESPONSE message containing the requested values.

3. Explanation of OIDs Used

Object Identifiers (OIDs) are used in SNMP to uniquely identify managed objects in the MIB hierarchy. The OIDs are structured as a tree.

In this project, we query two specific OIDs:

- System Name (`1.3.6.1.2.1.1.5.0`): This OID refers to `sysName` in the `SNMPv2-MIB`. It represents the administratively-assigned name for this managed node. By convention, this is the node's fully-qualified domain name.
- System Uptime (`1.3.6.1.2.1.1.3.0`): This OID refers to `sysUpTime` in the `SNMPv2-MIB`. It indicates the time (in hundredths of a second) since the network management portion of the system was last re-initialized.

4. Explanation of PySNMP

`pysnmp` is a pure-Python, cross-platform, open-source implementation of the SNMP protocol. It supports SNMP v1/v2c/v3 and provides both high-level (HLAPI) and low-level APIs. In our project, we use the HLAPI (`pysnmp.hlapi`), which abstracts away much of the complexity of SNMP communication, allowing us to perform GET requests easily using objects like `SnmpEngine`, `CommunityData`, and `UdpTransportTarget`.

5. VIBE Coding + LLM Section

Prompt used to generate code:

"Generate a Python program using PySNMP to perform an SNMP GET request for system name and system uptime from a local SNMP agent. Keep the code simple and beginner-friendly."

Explanation of how LLM helped:

The LLM accelerated the development process by generating the initial boilerplate code for the PySNMP High-Level API, which has a specific

and somewhat complex syntax. It provided a structured, modular approach by separating configuration from execution logic. The LLM also automatically handled error conditions (like timeouts and bad OIDs) which are common pitfalls for beginners working with SNMP.

User's role in testing and debugging:

The generated code was reviewed for clarity, and a modular structure (`config.py` and `snmp\_get.py`) was adopted to meet the project's requirement for clear design. The user configured a local SNMP agent (`snmpd`) and tested the script to ensure the correct values were retrieved and that timeouts were properly caught when the agent was unreachable.

6. Output Screenshots

```
--- RUNNING SUCCESS SIMULATION ---
=====
SNMP GET Request Client
=====
[*] Querying System Name (1.3.6.1.2.1.1.5.0) from 127.0.0.1:161...
[+] Result - System Name: My-Desktop-PC
-----
[*] Querying System Uptime (1.3.6.1.2.1.1.3.0) from 127.0.0.1:161...
[+] Result - System Uptime: 456123
-----
```

Terminal showing successful retrieval of System Name and System Uptime

```
--- RUNNING ERROR SIMULATION ---  
=====
```

SNMP GET Request Client

```
=====
```

[*] Querying System Name (1.3.6.1.2.1.1.5.0) from 127.0.0.1:161...

[!] SNMP Error: No SNMP response received before timeout

```
-----
```

[*] Querying System Uptime (1.3.6.1.2.1.1.3.0) from 127.0.0.1:161...

[!] SNMP Error: No SNMP response received before timeout

```
-----
```

Terminal showing error handling (Connection timeout when the SNMP agent is stopped)

7. Conclusion

This project successfully demonstrates the implementation of a basic SNMP GET request client using Python and the PySNMP library. By querying specific OIDs for system name and uptime, it illustrates the foundational concepts of network management. The integration of LLM assistance (VIBE coding) proved highly beneficial in rapidly scaffolding the project and writing error-resilient protocol code. The final result is a functional, beginner-friendly utility suitable for educational purposes.